

API-Based Data Integration

1. INTRODUCTION

The API-Based Data Integration Project is designed to demonstrate an automated data integration pipeline that extracts real-time weather information from a public REST API, processes and validates the extracted data, and stores the processed information in a structured data table using Databricks.

The project uses the Open-Meteo Weather API as the external data source. Weather information is retrieved for multiple cities and transformed into a structured format using Python and PySpark. Data quality checks are performed to identify duplicate or invalid records before loading the data into a Delta table.

The pipeline is implemented in Databricks and automated using a scheduled Databricks Job. The job is configured to execute daily at 09:00 AM IST, allowing the data integration process to run automatically without requiring manual execution.

The project demonstrates the complete data integration workflow:

API → Data Extraction → Transformation → Data Validation → Delta Storage → Automated Scheduling

2. PROJECT OBJECTIVE

The main objective of this project is to build a reliable and reusable data integration pipeline for collecting weather data from a public API and storing it in a structured format.

The key objectives of the project are:

- To integrate a public REST API with Databricks.
- To extract weather data for multiple cities.
- To transform the API response into a structured dataset.
- To perform data quality and validation checks.
- To store the processed data in a Delta table.
- To automate the pipeline using a scheduled Databricks Job.
- To ensure that the pipeline can execute regularly without manual intervention.

3. API INTEGRATION & DATA SOURCE

The pipeline uses the Open-Meteo Weather API, a public REST API, as the external data source. The API provides current weather information based on geographic coordinates.

The pipeline retrieves the following weather attributes:

- Temperature
- Relative Humidity
- Wind Speed
- Weather Code
- Observation Time
- City

Weather data is collected for multiple cities, including **Visakhapatnam, Hyderabad, Chennai, Bangalore, Mumbai, and Delhi.**

The API response is received in JSON format and is then processed using Python and PySpark for further transformation and validation.

OPEN-METEO API INTEGRATION CODE

```
def fetch_weather_data(city, latitude, longitude):  
  
    API_URL = "https://api.open-meteo.com/v1/forecast"  
  
    params = {  
        "latitude": latitude,  
        "longitude": longitude,  
        "current": "temperature_2m,relative_humidity_2m,wind_speed_10m,weather_code",  
        "timezone": "Asia/Kolkata"  
    }  
  
    try:  
        response = requests.get(  
            API_URL,  
            params=params,  
            timeout=30  
        )  
  
        response.raise_for_status()  
  
        data = response.json()
```

4. DATA TRANSFORMATION & QUALITY VALIDATION

After extracting the weather data from the API, the response is transformed into a structured format using Python and PySpark. The required fields are selected and prepared for storage. Data quality checks are performed before loading the data to ensure that the dataset is reliable. The pipeline validates duplicate records and checks whether temperature, humidity, and wind-speed values are within valid ranges.

The validation results confirmed that the extracted dataset contained no duplicate or invalid weather records.

Data Quality Results

- Duplicate records: **0**
- Invalid temperature records: **0**
- Invalid humidity records: **0**
- Invalid wind-speed records: **0**

DATA TRANSFORMATION & QUALITY VALIDATION

```
from pyspark.sql.functions import col

quality_summary = {
    "total_records": pipeline_spark_df.count(),

    "null_city": pipeline_spark_df.filter(
        | col("city").isNull()
    ).count(),

    "null_temperature": pipeline_spark_df.filter(
        | col("temperature_c").isNull()
    ).count(),

    "null_humidity": pipeline_spark_df.filter(
        | col("humidity_percent").isNull()
    ).count(),

    "null_observation_time": pipeline_spark_df.filter(
        | col("observation_time").isNull()
    ).count(),
}
```

> [See performance \(8\)](#) ✓ Checked for improvements

```
total_records: 6
null_city: 0
null_temperature: 0
null_humidity: 0
null_observation_time: 0
invalid_temperature: 0
invalid_humidity: 0
invalid_wind_speed: 0
```

5. DATA STORAGE

After successful transformation and validation, the weather data is stored in Databricks using a Delta table.

A database named **weather_db** was created, and the processed weather records were stored in the **weather_data** table. PySpark was used to define and process the structured dataset before loading it into Delta storage.

The pipeline successfully loaded **6 weather records** into the table.

Database: weather_db

Table: weather_data

Records Loaded: 6

DELTA TABLE DATA LOADING USING MERGE OPERATION

```
%sql
MERGE INTO weather_db.weather_data AS target
USING weather_pipeline_batch AS src
ON target.city = src.city
AND target.observation_time = src.observation_time

WHEN MATCHED THEN UPDATE SET
    target.country = src.country,
    target.latitude = src.latitude,
    target.longitude = src.longitude,
    target.temperature_c = src.temperature_c,
    target.humidity_percent = src.humidity_percent,
    target.wind_speed_kmh = src.wind_speed_kmh,
    target.weather_code = src.weather_code,
    target.weather_category = src.weather_category,
    target.`source` = src.`source`,
    target.load_timestamp = src.load_timestamp,
    target.observation_date = src.observation_date
```

	num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
1	6	0	0	6

WEATHER_DB TABLE

11:32 AM (37s)14

%sql
SELECT * FROM weather_db.weather_data;

> See performance (1)✓ Checked for improvements

	city	country	latitude	longitude	observation_time	temperature_c
1	Visakhapatna...	India	17.68014	83.204254	2026-09-04T09:00:00.000+00:00	30.6
2	Visakhapatna...	India	17.68014	83.204254	2026-09-03T10:30:00.000+00:00	32.7
3	Hyderabad	India	17.398945	78.457085	2026-09-04T09:00:00.000+00:00	27.1
4	Bangalore	India	12.970123	77.56364	2026-09-04T09:00:00.000+00:00	24.1
5	Hyderabad	India	17.398945	78.457085	2026-09-03T10:30:00.000+00:00	26.7
6	Bangalore	India	12.970123	77.56364	2026-09-03T10:30:00.000+00:00	26.7
7	Mumbai	India	19.086115	72.85291	2026-09-04T09:00:00.000+00:00	27.2
8	Chennai	India	13.110721	80.2459	2026-09-04T09:00:00.000+00:00	31.4
9	Mumbai	India	19.086115	72.85291	2026-09-03T10:30:00.000+00:00	27.4
10	Chennai	India	13.110721	80.2459	2026-09-03T10:30:00.000+00:00	34

	city	country	latitude	longitude	observation_time	temperature_c
1	Visakhapatna...	India	17.68014	83.204254	2026-09-04T09:00:00.000+00:00	30.6
2	Visakhapatna...	India	17.68014	83.204254	2026-09-03T10:30:00.000+00:00	32.7
3	Hyderabad	India	17.398945	78.457085	2026-09-04T09:00:00.000+00:00	27.1
4	Bangalore	India	12.970123	77.56364	2026-09-04T09:00:00.000+00:00	24.1
5	Hyderabad	India	17.398945	78.457085	2026-09-03T10:30:00.000+00:00	26.7
6	Bangalore	India	12.970123	77.56364	2026-09-03T10:30:00.000+00:00	26.7
7	Mumbai	India	19.086115	72.85291	2026-09-04T09:00:00.000+00:00	27.2
8	Chennai	India	13.110721	80.2459	2026-09-04T09:00:00.000+00:00	31.4
9	Mumbai	India	19.086115	72.85291	2026-09-03T10:30:00.000+00:00	27.4
10	Chennai	India	13.110721	80.2459	2026-09-03T10:30:00.000+00:00	34

LOAD DATA DELTA TABLE

▶

✓ Yesterday (4s)

17

```
log_df.write \
  .format("delta") \
  .mode("append") \
  .saveAsTable("weather_db.pipeline_logs")

print("Pipeline log saved successfully")
```

> [See performance \(1\)](#)

✓ Checked for improvements

Pipeline log saved successfully

6. JOB AUTOMATION & SCHEDULING

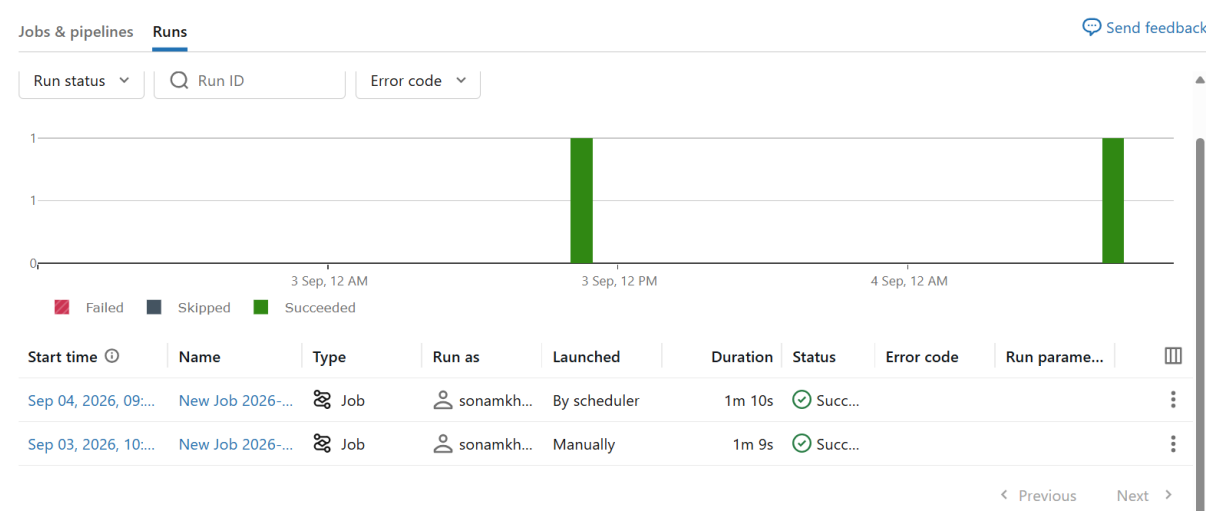
The completed pipeline was configured as a Databricks Job to automate its execution. The job uses the API_Data_Integration_Pipeline notebook with Serverless compute. The pipeline was first executed manually to verify that the complete workflow was working successfully. The manual job run completed with a **Succeeded** status. A daily schedule was then configured to automatically execute the job at **09:00 AM IST**.

Job Details

- **Job ID:** 95513066222439
- **Notebook:** API_Data_Integration_Pipeline
- **Compute:** Serverless
- **Manual Run Status:** Succeeded
- **Scheduled Run Status:**Succeeded
- **Schedule:** Daily at 09:00 AM IST

SUCCESSFUL DATABRICKS JOB EXECUTION AND DAILY SCHEDULE

Jobs & Pipelines



7. CONCLUSION

The PI-Based Data Integration Project successfully implemented an automated weather data pipeline using Python, REST API, PySpark, and Databricks.

The pipeline extracts weather data from the Open-Meteo API, transforms and validates the data, and stores the processed records in a Delta table. Data quality checks were successfully completed with no duplicate or invalid records.

The pipeline was also configured as a scheduled Databricks Job to run automatically every day at 09:00 AM IST.

Overall, the project demonstrates a complete and automated data integration workflow from API extraction to data storage and scheduled execution.